1 Reconstruction IV: Dynamic Analysis

In the previous sessions, we have looked at the source code, we have looked at version control, and we saw that interesting information is available there.

1.0.1 However, we have now to face the elephant in the room: the *running system itself*.

We can **not** *not look* at it in our attempts to understand the system's architecture, even if, this is going to be the most challenging aspect.

1.0.2 There are multiple ways in which we can analyze a running system

- add *ad hoc* **logging** statements to the system
- ***instrument*** the code that is being executed by using reflection
- **monitor network traffic** for distributed systems

We will today discuss how can this kind of information be used in architecture recovery.

1.1 Limitations of Static Analysis

1.1.1 Scenario: You want to run a dead code detection analysis

Let us assume that we want to discover whether a given system has code that is not used. This happens quite often actually.

- How are we going to do it with static analysis?
- What are the limitations of static analysis in this particular problem?
  - code might look *connected* to the rest of the call graph but *never be called* in practice
  - code might look *disconnected* but be *called using reflection*, or be an entry point for testing, or ...

1.1.2 Limitations of static analysis

Some of the limitations of static analysis:

1.1.2.1 Overestimates some relationships One such example is **runtime polymorphism** - from studying the source code one can not know which of the many alternative implementations is actually used

1.1.2.2 Some information is only really available at runtime

- dynamic code evaluation (e.g. `eval`)
- code that is dependent on user-driven input
- usage of reflection

1.1.2.3 Can not provide information execution properties

- E.g., memory consumption, running times, and timing might be architecturally relevant

1.2 What Is Dynamic Analysis?

1.2.1 A technique of program analysis that consists of observing the behavior of a program while it is executing.

Dynamic analysis collects **execution traces** = records of the sequence of actions that happened during an execution.

If we recorded the functions in the system that get executed we could improve the precision of our dead code detection scenario of earlier:

> If we had information from the execution of the system we could exclude some of the **false positives** if we see that they are used at runtime.

2 How To Instrument Systems for Analysis?

The key activity in dynamic analysis is **instrumenting the system**, that is, modifying the system such that we can extract information from its execution.

There are multiple such methods. We'll discuss three today:

2.1 Logging

Adding log statements in the program can help collect traces of its execution.

2.1.1 Benefits

The benefits of this approach are: #### Allows surgical precision

We can add log statements only where relevant (e.g. if I want to investigate the calls between two particular classes, you can add log statements only in those classes.

2.1.1.1 Technology is straightforward to use

It's even too much to call it technology :) You use `console.log()`, `print()`, `logging.log()`, etc.

2.1.2 Limitations

2.1.2.1 Invasive

Implies changing the program and adding log statements everywhere.

Usually we want to log extensively so there is a lot of manual work needed.

2.1.2.2 Tracking logs in distributed systems is challenging Why do we care about distributed systems? Because everybody and their dog jumped on micro-services.

The solution for logging in the context of a distributed system is a combination of techniques listed below:

2.1.2.2.1 Technique #1: tracing the sequence of messages in a distributed system The simplest way to tracking logs in a distributed system is to add timestamps in every logging statement

The limitation of this approach can be hit at limits when the nodes in the system have their clocks desynchronized.

The precise approach is tracking requests as they propagate through multiple nodes in the system

This more involved approach is called **distributed tracing** - tracking requests as they propagate through multiple microservices by adding a unique request ID or trace ID to every message

2.1.2.2.2 Technique #2: Centralized logging Logging for distributed systems, e.g. services and micro-services, requires the collection all the logs in one place.

2.2 Dynamic Behavior Modification

What if we could modify every method call to log it's call automatically. Not by adding a log statement at the beginning of every method, that would not scale.

We would rather modify the way the program is executed such that we can *automatically* log the fact that a given method is called.

Then we could re-create a sequence diagram of the whole execution of a program!

Let's look at a few appraoches.

2.2.1 Approach #1: Using Reflection

2.2.1.1 Reflection is the ability of a program to manipulate as data something representing the state of the program during its own execution In some languages it's easier to do (e.g. Ruby, Python) than in others (Java).

There are two kinds of reflection:

2.2.1.1.1 1: Introspection is the ability for a program to observe and therefore reason about its own state. E.g. listing the methods in a class in Python

- Every class has a `__dict__` that is a dictionary mapping the names of it's attributes to the objects that represent it (e.g. `Exception.__dict__.items()`)
- A method can be detected because it has the `__call__` attribute (e.g. 'hasattr(object, '**call**'))

Putting the two together, we can define:

```
def methods_in_class(cls):
    return [
        (name, object)
        for (name, object)
            in cls.__dict__.items()
        if hasattr(object, '__call__')]
```

2.2.1.1.2 2: Intercession is the ability for a program to modify its own execution state or alter its own interpretation or meaning. E.g. replacing all the methods in a class with decorators that print call information before executing the original behavior in Python can be done in a few steps:

Step 1: Define a decorator function

That decorator could simply log the function call before delegating to the function, e.g.

```
def log_decorator( function ):
    def decorated( *args, **kwargs ):
        print (f'I have been called: {function}')
        return function ( *args,**kwargs )
    return decorated
```

Step 2: Define a function to decorate all the methods in a class

This can be done by reusing our `methods_in_class` function from above:

```
def decorate_methods( cls, decorator ):
    methods = methods_in_class(cls)
    for name, method in methods:
        setattr( cls, name, decorator ( method ))
```

Step 3: Do the actual decoration

```
from zeeguu.core.model import User
decorate_methods(User, log_decorator)

u = User.find_by_id(534)
u.bookmark_count()

# to see even further one can instrument also third party libraries!
from sqlalchemy.orm.query import Query
decorate_methods(Query, log_decorator)
```

Step #4: Using introspection to detect the calling site

In the example above we used introspection to figure out the methods in a class. We can also use introspection to query the current state of the Python call stack with the help of the `inspect` package.

```
import inspect

def caller():
    callee()

def callee():
    print(inspect.stack()[1].function)

caller()
```

Challenge: can you plug this solution in the `log_decorator` for a more complete execution trace?

2.2.1.2 Function Wrappers In the previous section, the `log_decorator` is what is called a **function wrapper** == a pattern inspired from the Decorator design pattern:

- A function *wraps* another function in order to ...
  - perform some *prologue* and *epilogue* tasks, or to
  - optimize (e.g. cache results)
- ... while the *wrapper* is *fully* compatible with the wrapped function so it can be used instead

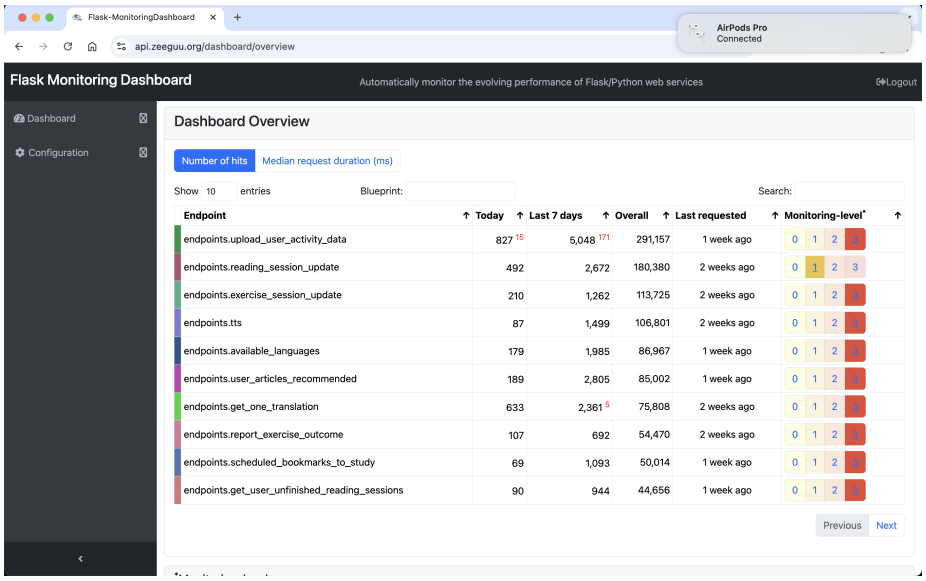
2.2.1.2.1 Advantages of Wrappers

- make it **easy to automate** (e.g. you could iterate through all the modules and all the classes in Zeeguu using reflection, and deploy a wrapper on every function)

2.2.1.2.2 Disadvantages of Wrappers

- they introduce an **overhead** (but then, so do all code instrumentation techniques)
- they require to be deployed on **live** objects
- must be in the same process as the instrumented code

2.2.1.2.3 Application of Function Wrappers A performance monitor implemented at RUG and ITU across several BSc and MSc theses:

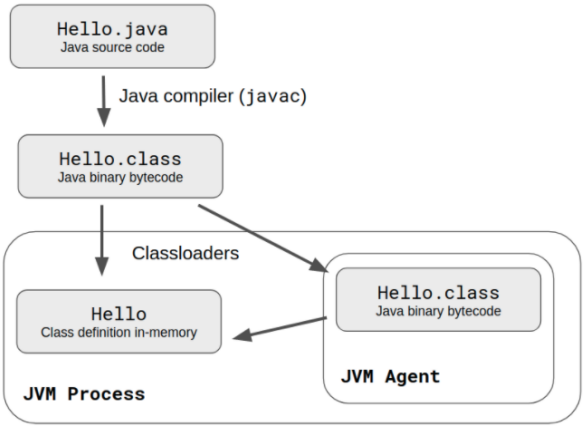


https://github.com/flask-dashboard/Flask-MonitoringDashboard

2.2.2 Runtime Instrumentation

2.2.2.1 RT is a technique that modifies the generated code representation in order to avoid modifying the actual code. Example: instrumenting the bytecode of Java programs can be done with a tool called the Java Agent. This is possible because:

- Java programs are compiled into bytecode
- Bytecode is executed on the JVM
- *Instrumenter* provides a Java Agent (via command line argument -javaagent) that modifieds the bytecode before it being executed



Advantage: - JVM bytecode instrumentation works for multiple languages

2.3 Network Traffic Analysis

Not considered as part of *traditional dynamic analysis* but becomes more relevant

- useful for service oriented architectures
- monitors the messages on the wire
- powerful approach for reverse engineering services

Read: https://danlebrero.com/2017/04/06/documenting-your-architecture-wireshark-plantuml-and-a-repl/

Note: An approach like this would be a great starting point for a thesis.

3 How to Run the Instrumented Systems?

3.1 Running the code itself might pose challenges

- Configuration
- Dependencies
- Unwritten rules
- Some systems don't have a clear entry point (e.g. libraries)

Helpful practices that make running code easier:

- continuous integration
- containerization
- infrastructure as code

3.2 ## Which Scenarios to Run from the System?

- Run the unit tests if they exist
- Exercise individual "features"

  > A feature is a realized functional requirement of a system. [...] an observable unit of behavior of a system triggered by the user [Eisenbarth et al., 2003].

4 Limitations of Dynamic Analysis

4.1 Limited by execution coverage

> Dynamic analysis is **related to testing and shares the same disadvantages**.
>
> All the conclusions you draw are valid only with respect to the given input. When it comes to architecture, however, we are generally interested in all possible behavior. (Koshcke, ***What architects should know about reverse engineering and reengineering***)

A program does not reach an execution point... => no data (e.g. Word but user never uses the print option)

4.2 Can slow down the application considerably

Do you know how many function calls are in a second? Imagine duplicating them because of the print statements. And writing to file after each. That's going to slow down painfully your application!

4.3 Can result in a large amount of of data

A few seconds of execution can result in GB of data for complex systems

Benefits of dynamic analysis for Architecture Recovery

Dynamic analysis is an essential **complement for static analysis** for dependency extraction.

The information extracted from dynamic analysis **can be aggregated** along the same axes as static.

One can do cross-language dependency extraction with the help of dynamic analysis. *Can you think of examples and how would you do this?*

5 Challenges for You

Extract dynamic dependencies from your case study system.

- can you create a wrapper that traces method calls (both the caller and the callee?)
- fully qualified names of the caller method

6 Bibliography

[1]) What architects should know about reverse engineering and reengineering R. Koschke, In 5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05)_(pp. 4-10). IEEE.

7 Further Reading

Function Wrappers - https://wiki.python.org/moin/FunctionWrappers - Wrappers to the Rescue: http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.18.6550&rep=rep1&type=pdf